

자연어 처리

영화리뷰 데이터를 이용한 감성분석

조장 : 최재철

조원 : 박정환, 최혜정

텍스트 분류

목차 소개

01

데이터 소개

데이터 수집 방법
속성 및 컬럼 확인

02

분석 절차

자연어 처리
분석 목표

03

데이터 전처리

영어, 특수문자 등 제거
불용어 처리

04

모델 학습

TF-IDF 분석
BERT, Transformers

05

성능 평가

loss, accuracy
f1 score

데이터 소개



#영화리뷰

#영화정보

#박스오피스

KOBIS 23년도 영화상영 정보 및 네이버 영화 관람평을 크롤링 하여 데이터를 수집하였습니다.



- 영화관입장권통합전산망이 제공하는 오픈API 활용.
- 박스오피스, 영화, 영화사, 영화인 정보를 조회할 수 있으며 XML과 JSON형식으로 응답 받을 수 있음.



```
> pip install selenium
```

- 크롬드라이버(구글에서 제공하는 컴퓨터가 크롬 웹 브라우저를 다룰 수 있도록 해주는 프로그램) 이용.
- 버튼을 클릭 하거나 문자를 입력하는 등의 일을 Selenium 으로 실행 가능

데이터 소개

부정리뷰 예시 :

1 점 리뷰 :
 >>> 진짜 드럽게 지루하고 유치하고 재미없음 하
 >>> 이번년도 지금까지는 워스트 영화
 >>> 기대에 현참 못미치네요 ㅠㅠ
 >>> 이해는 하지만 이견 좀
 >>> 이거 진짜 극한 직업 벨로가 체질을 만들어낸 이병헌의 작품이 맞는가 후반부 신파와 국뽕의 골라보는 진짜 극장 나감까 오만번 정도 고민했다 캐스터 대사 쓰면서 진짜 이게 맞나 라는 생각 안 들었을까 그냥 극혐이라는 단어 말고는 생각나지 않는 공포의 시간이었다 코미디 영화 웃긴 사람만 웃긴 이병헌 감독 특유의 유머러스함을 전작 주연 배우들에 비해 소화해내지 못하는 아이유와 박서준 그렇지만 그렇지만서도 대중적 호감도가 높은 아이유 박서준이 아니었다면 이 영화는 지금보다 훨씬 혹평을 받았을 것이다 이 줄작이 진짜 이병헌 감독의 작품이라고 믿고싶지 않다 거대 자본으로부터의 입김이 너무 셧다고 생각하고 싶다
 >>> 와 돈 원도 아까운 영화임아 정말 시간 너무 아깝다 ㅇ하
 >>> 액역도 색이 밋밋하고 마동석이 신인 세계관인가 ㅋㅋㅋㅋ 뭐 그냥 주먹질 몇 번 하면 다 쓰러지네 ㅋㅋ
 >>> 돈 주고 봐도 아깝지만 돈 안주고 봐도 시간이 아깝다 러닝타임도 길어서 더 괴로웠다 이런 작품을 보고싶지않다
 >>> 그냥 정도 아깝다 이걸 돈주고 사는 인간이 나왔다니 광고보다 더한 연출 스토리 뭐하나 제대로 된게 없다 이정도면 돈주고 영화본 사람들에게 대국민사과해야한다 진짜 어지간해서 평점 짜게 안주는데 이걸 너무 심하다 이런 영화가 개봉된다는 것은 이런 극본을 보고 투자하는 투자자들이 있다는거겠지 이런 영화가 계속해서 나온다는걸 보면 한국 영화계엔 미래가 없다 적당선에서 짜를 생각을 해야지 좋은 배우들 가져다 쓰고 이런 쓰레기를 만들어내는걸 보면 감독은 천재가 아닐 수 없다
 >>> 이게 대체 무엇일까요

긍정리뷰 예시 :

10 점 리뷰 :
 >>> 흥미진진하게 보기 좋았음 이걸로 효도함
 >>> 년간 그들을 사랑한 이유를 다시 한번 깨닫게 되었다 피보다 진한 명의 형제애와 팬들을 대하는 사랑과 진심 데뷔초에 국민그룸으로 전성기 탑을 찍었음에도 늘 초심을 잃지 않고 성장하는 그룸 그들도 몰랐고 우리도 몰랐지만 주년에 콘서트 실행 영화가 이렇게 개봉하고 관심을 얻고 팬들이 많아지는 전무후무한 아이돌그룸임 는 살아있는 전설이고 앞으로가 더 기대되는 성장형 가수라는게 너무 자랑스럽다 팬자오디도 년간 아주 현결감이 그들을 서포트하고 사랑하고 화력을 뽐내고 서로에게 마스터피스같은 존재 내가 팬지라는게 너무 너무 자랑스럽다
 >>> 한국인 이라면 특히 한국 남자라면 꼭 봐야하는 영화라고 생각합니다 유지컬 영화로서 완성도도 높았고 정성화씨 연가력 가창력도 뛰어납니다 그보다도 안중근 장군의 삶이 전해주는 묵직한 때문에 영화관을 떠난 다음날도 여운이 남고 한국인으로서 또 남자로써의 삶을 돌아보게 됩니다 또 같은 종교인으로써의 고민도 느껴지면서 대장부로 태어나 제운 뜻을 위해 전념하며 살지 못하는 자신을 반성하였습니다 안중근 장군에 대해 네이버 검색하면서 첫째 아들은 살에 일제에게 독살 당하고 남겨진 가족들이 일제에게 갖은 고통을 겪고 남은 둘째 아들이 일제에게 변절했다는 사실을 알게 되었습니다 이순신 장군과 함께 생각만해도 가슴이 먹먹해 지는 이름 안중근 장군입니다
 >>> 부 기다린보람이 있음 **
 >>> 정우성의 인생 연기 행파봉 드립니다 그날 얼마나 어처구니 없는 일이 일어난 건지 분해서 눈물이 난다
 >>> 갓태인 것도하 갓복동제임스그들의 매력적인 마스크와 연기는 나를 설레게했다
 >>> 와기대없이 봐서 그럴까요너무 재미있네요생각없이 크게 웃고웃음포인트 많음잔잔한 감동도 있어요함께 본 엄마도 간만에재미있는 한국영화라고배역을 캐스팅도 훌륭합니다
 >>> 아름다운 영화영화다운 영화
 >>> 세월을 거쳐 온 이 영화를 극장에서 처음 보았다 장국영은 여전히 이 시대에 존재하는 배우다 아름다운 새는 영원히 떨어지지 않을 것이다
 >>> 존재에 대해 깊이 고민하는 영화

네이버 관람평

movie_reviews (영화리뷰)

	항목명	설명
1	raw_data	스크래핑 원본 데이터
2	movieNm	영화명(국문)
3	별점	1~10점
4	review	리뷰데이터
5	작성자	네이버아이디(뒤의 4자리 '*')
6	작성일	YYYY.MM.DD HH:MM
7	추천	리뷰 추천 수
8	반대	리뷰 반대 수
9	movieCd	영화의 대표코드

- 영화의 대표코드(movieCd)는 박스오피스 순위의 코드와 동일한 코드
- 최종적으로 이용하는 데이터는 영화 별점과 리뷰 데이터.

PROCESS

자연어 처리

자연어에서 의미 있는 정보를 추출하여 활용 (Natural Language Processing, NLP)
예를 들면 문장에서 다음에 나올 단어의 확률을 구하며, 단어, 문장, 문단에 적용한 패턴들을 인식하는 등의 과정.

LDA

토픽모델링

TF-IDF

핵심단어 확인
유사도 분석과 SVM

LSTM,GRU

순환 신경망 기반
분류/예측 모델

BERT

트랜스포머 기반
분류/생성 모델



긍정/(중립)/부정

이진분류, 다중분류

데이터 전처리

*NLP의 주요 전처리 방법

문장단위로 구분

KoNLPy의 Kkma 문장구분

- 긴 문장을 작은 단위의 문장으로 나누는 작업 가능.

띄어쓰기

PyKospacing의 spacing

- 잘못 된 띄어쓰기에 대한 전처리 가능.

✓ 영문, 특수문자 제외

`re.sub(r"[^ㄱ-ㅣ가-힣\s]", "", x)`

- 정규표현식을 활용하여 영어문자, 특수문자 등을 제외하고 한글데이터만 사용.

✓ 형태소 분석

KoNLPy의 Okt를 이용하여 분석

- 트위터에서 만든 오픈소스 한국어 처리기인 twitter-korean-text를 이어서 만드는 프로젝트

✓ 불용어 처리

자주 등장하지만 의미가 없는 용어 처리

- 조사, 접미사 같은 불용어 정리
- 토큰화 후에 조사, 접속사 등을 제거

데이터 전처리

한글과 공백을 제외하고 제거

```
df['review'] = df['review'].str.replace("[^ㄱ-ㅎㅏ-ㅣ가-힣 ]","", regex=True)
```

형태소 분석 & 불용어 제거

```
li = []
for sentence in tqdm(df['document']):
    tokenized_sentence = okt.morphs(sentence, stem=True) # 토큰화, 텍스트를 형태소 단위로 나눔
    stopwords_removed_sentence = [word for word in tokenized_sentence if not word in stopwords]
    li.append(stopwords_removed_sentence) # stopwords : 불용어 리스트
```

타겟 변수 생성

별점 정보를 활용하여 target_1, target_2 변수 생성

target_1, 이진분류

- 8 이상 : 긍정 (1)
- 8 미만 : 부정 (0)

```
df['target_1']=np.where(df['별점']>=8,1,0)
```

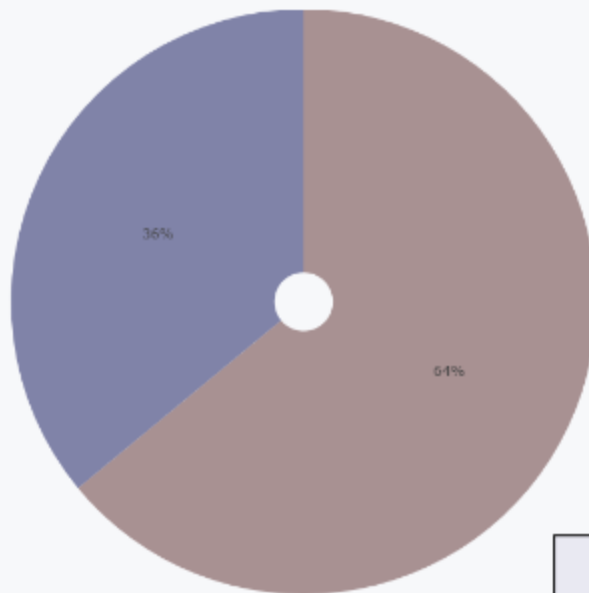
target_2, 다중분류

- 별점 8 이상 : 긍정 (2)
- 6 이상 8 미만 : 중립 (1)
- 6 미만 : 부정(0)

```
conditions=[ df['별점']>=8,
              (df['별점']>=6) & (df['별점'] < 8), df['별점'] < 6 ]
choices=[2,1,0]
df['target_2']=np.select(conditions,choices,default='NA')
```

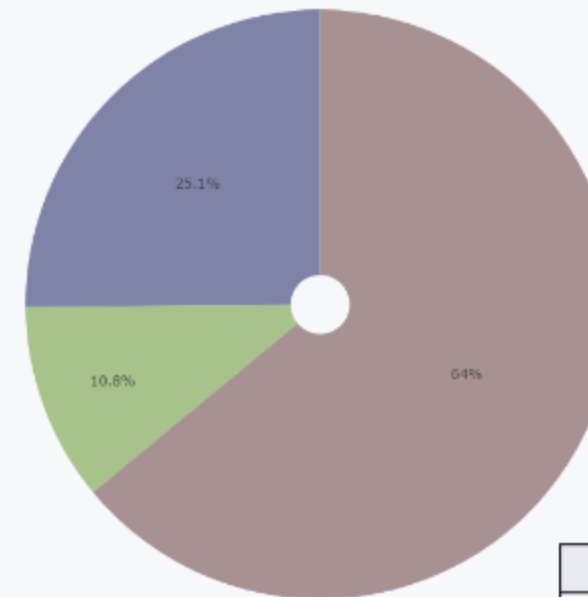
타겟 분포

이진분류 (긍정, 부정)



	구분	건수
1	긍정	32,404
2	부정	18,217

다중분류 (긍정, 중립, 부정)



	구분	건수
1	긍정	32,404
2	부정	12,728
3	중립	5,489

모델 학습, topic

문서 집합의 추상적인 "주제"를 발견하기 위한 통계적 모델 중 하나로,
텍스트 본문의 숨겨진 의미구조를 발견하기 위해 사용되는 텍스트 마이닝 기법

잠재 디리클레 할당(LDA; Latent Dirichlet Allocation)이란?

> 단어가 특정 토픽에 존재할 확률과 문서에 특정 토픽이 존재할 확률을 결합확률로 추정하여 토픽을 추출하는 방법

```
from gensim.models.ldamodel import LdaModel
```

```
# 데이터 세트 코퍼스의 단어 단위 표현에 대한 LDA(Latent Dirichlet Allocation) 모델을 훈련합니다.
# LdaModel은 지정된 매개변수를 사용하여 데이터세트_corpus_bow에 적용됩니다.
# - num_topics: 말뭉치에서 식별할 주제 수(이 예에서는 2으로 설정)
# - id2word: 토큰을 정수 ID로 매핑하는 사전(이전에 생성된 데이터 세트_dictionary 사용)
# - random_state: 재현성을 위한 시드
lda_bow = LdaModel(dataset_corpus_bow, num_topics=10, id2word=dataset_dictionary,
random_state=42)
```

(10개의 토픽 추출)

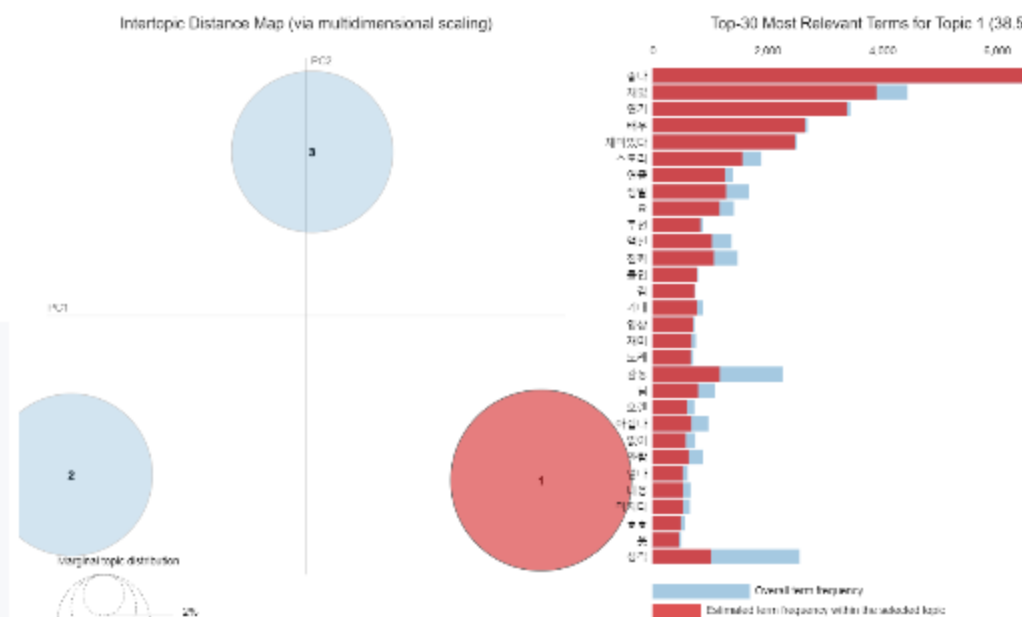
```
Topic 0: ['모든', 'ㅠㅠ', '역시', '봐도', '눈물이']
Topic 1: ['잘랐습니다', '모르고', '시간이', '좋았음', '편']
Topic 2: ['재밌게', '생각보다', '가장', '봤는데', '중']
Topic 3: ['최고의', '여운이', '남는', '올해', '봐야']
Topic 4: ['마지막', '아름다운', '그럭저럭', '걸', '눈물']
Topic 5: ['재미있게', '다시', '재미있어요', '영화입니다', '번']
Topic 6: ['무난하게', '보았습니다', '감독의', '영화네요', '위한']
Topic 7: ['이렇게', '영화에', '것을', '재미있었어요', '있어요']
Topic 8: ['진짜', '연기', '좋았어요', '연기가', '좋았습니다']
Topic 9: ['없는', '영화였습니다', '이야기', '대한', '내가']
```

모델 학습, topic

LDA 토픽 모델링 시각화.

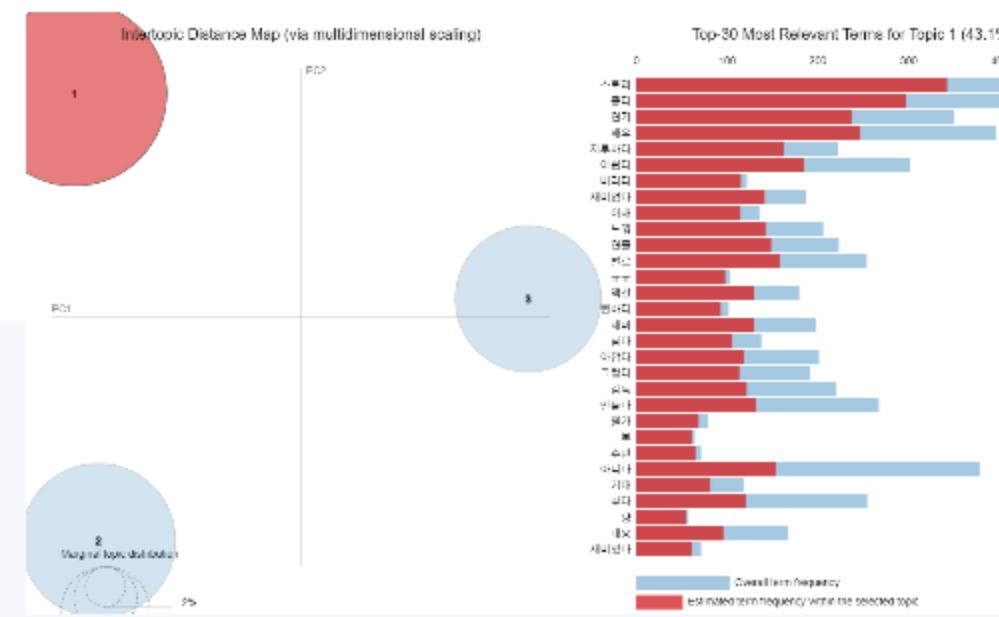
분포가 잘 떨어져 있다는 것은 각 토픽이 서로 다른 단어들을 가지고 있는 것

긍정



- 1) 액션, 몰입, 연출
- 2) 인생, 사랑, 아름답다
- 3) 실화, 역사, 감동

부정



- 1) 연기, 아쉽다, 별로, 뻔하다, 지루하다
- 2) 모르다, 무섭다, 유치하다
- 3) 노잼, 안타깝다, 아니다

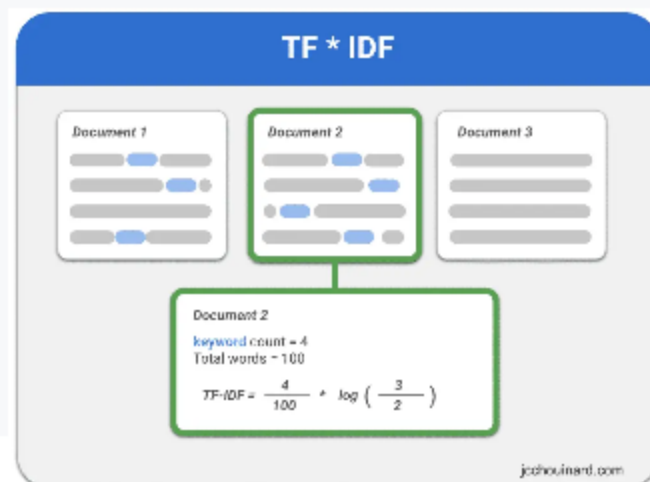
모델 학습, tf-idf

TF-IDF(term frequency-inverse document frequency)란
코퍼스(corpus, 문서집합)에서 한 단어가 얼마나 중요한지를 수치적으로 나타낸 가중치

(1) $tf(d,t)$: 특정 문서 d 에서의 특정 단어 t 의 등장 횟수
DTM이 각 문서에서의 각 단어의 등장 빈도를 나타내는 값이므로
 $tf(d,t)$ 와 같은 의미를 가진다.

(2) $df(t)$: 특정 단어 t 가 등장한 문서의 수.
특정 단어가 각 문서, 또는 문서들에서 몇 번 등장했는지는 관심을 가지지 않으며 오직 특정 단어 t 가 등장한 문서의 수에만 관심을 가집니다.

(3) $idf(d,t)$: $df(t)$ 에 반비례하는 수
DF에 역수를 취한 것



BoW (Bag of Words)

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
cv = CountVectorizer()  
cv.fit(data['review'])
```

> 각각의 문서에서 각각의 단어가 몇 번 등장했는지를 세서 행렬 형태로 표현
(문서를 벡터로 표현)

LogisticRegression()
0.68

LinearSVC()
0.65

로지스틱 회귀와 선형 SVM 모델의 성능을 계층적 교차 검증을 통해 평가하고, 각각의 평균 성능을 계산하여 출력.

모델 학습, tf-idf

TF



Frequency of a word
within the document

IDF



Frequency of a word
across the documents

×

즉, 단어의 빈도(Term Frequency)와 역 문서 빈도(Inverse Document Frequency)를 토대로, 특정 문서 내에 어떤 단어가 얼마나 중요한 지를 나타내는 통계적 수치.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

- `TfidfVectorizer.fit(text)`를 통해 `text`가 가지고 있는 모든 단어를 BoW로 구성하고, 이 단어들에 대해 Tf-idf 값을 계산한 뒤 각 단어의 인덱스 위치에 Tf-idf 값이 들어간 벡터 생성.

CountVectorizer와 같이 텍스트 데이터를 벡터 형태로 변환.

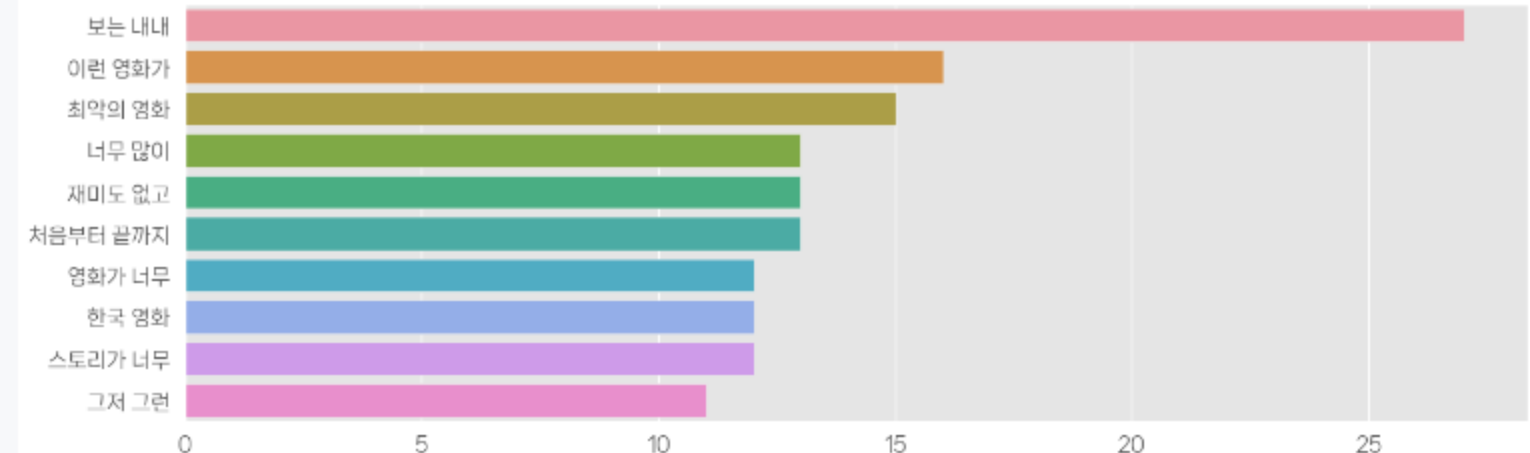
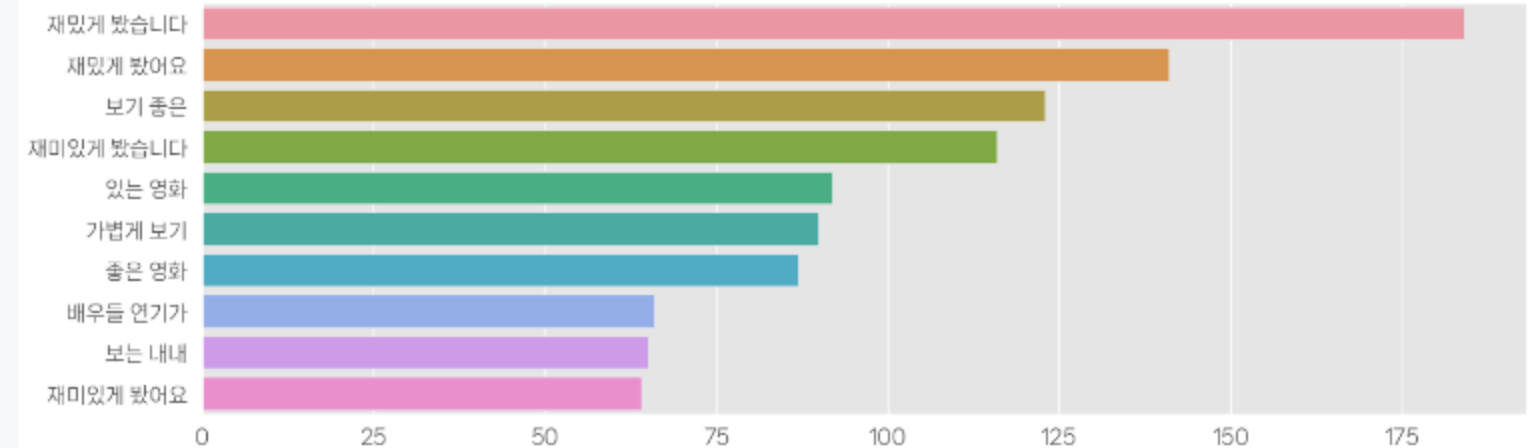
→ CountVectorizer는 단어의 발생 빈도만을 고려하여 문서를 벡터화하는 반면, TfidfVectorizer는 단어의 발생 빈도뿐만 아니라 해당 단어가 문서 집합에서 얼마나 중요한지를 반영하여 문서를 벡터화합니다.

리뷰 중요단어

긍정



부정



모델 학습, tf-idf

cross-validation(교차검증)

데이터를 train/test로 한 번만 나누는 것이 아닌, 여러 번 다른 방식으로 나누고, 나눈 train/test 데이터를 교차하여 선택하여 이를 토대로 모델의 성능을 평가하는 방법

K-fold

→ 클래스별 분포 고려 :



- 하이퍼파라미터나 모델의 최적화 시, 최적의 조건을 찾는 데에 활용될 수 있음.
- 한 번 나누어서 학습하는 것에 비해 일반화된 성능을 얻을 수 있음.

GridSearchCV

머신러닝에서 모델의 성능향상을 위해 쓰이는 기법중 하나입니다.
사용자가 직접 모델의 하이퍼 파라미터의 값을 리스트로 입력하면 값에 대한 경우의 수마다 예측 성능을 측정 평가하여 비교하면서 최적의 하이퍼 파라미터 값을 찾는 과정을 진행

→ 시간이 오래걸린다는 단점이 있음.

```
from sklearn.model_selection import GridSearchCV
```

```
scv_model = LinearSVC()

param_svm = { 'C' : [0.125, 0.5, 1, 2, 8] }

gscv_svm = GridSearchCV(estimator = scv_model
                        , param_grid = param_svm, scoring = 'f1'
                        , cv=StratifiedKFold(n_splits=5)
                        , refit=True, n_jobs=1, verbose=2)

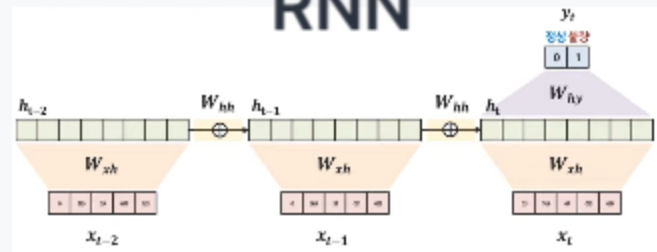
gscv_svm.fit(X_train_transform, data['label'])
model = gscv_svm.best_estimator_ # 최적의 파라미터로 모델 생성

# 최종 test 데이터 예측
pred = model.predict(X_test_transform)
pred = pd.DataFrame(pred, columns=['result'])
```

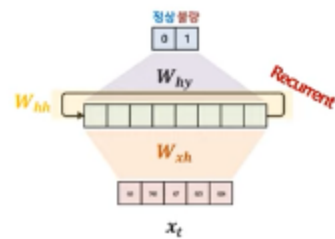
모델 학습, LSTM & GRU

순환 신경망은 사용자가 순차 또는 시계열 기반 데이터에 대한 예측 작업을 수행하고자 할 때 사용되는 인공지능망입니다. 이러한 딥 러닝 계층은 자연어 처리, 신경 기계 번역, 자동화된 이미지 캡션 작업 등과 같은 순서 또는 시간 문제에 일반적으로 사용됩니다.

RNN

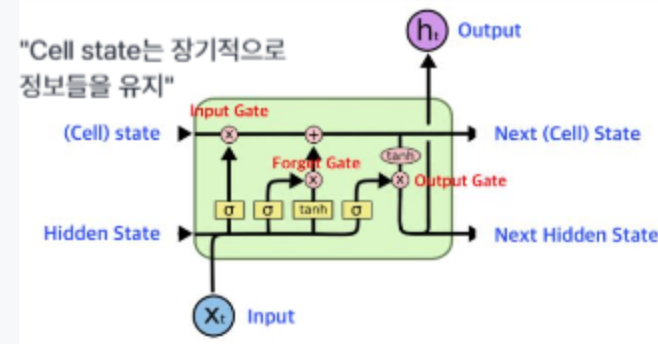


- RNN 구조의 핵심 개념 : t시점의 히든 정보는 t-1의 정보를 포함하고, t-1시점의 히든 정보는 t-2의 정보를 포함.
- t시점에서의 y를 예측하기 위해 t시점의 관측치와 히든벡터(레이어) 정보만 이용하지 말고, 과거의 정보를 이용합니다.



- 장기의존성 문제
- 시퀀스의 길이가 길어질 수록 과거 정보 학습에 어려움이 발생 (vanishing gradient)

LSTM



"Cell state는 장기적으로 정보들을 유지"

- LSTM의 핵심은 Cell State, Gate 라고 불리는 구조에 의해 정보가 추가되거나 제거됨.
- forget gate (f) : 과거 정보를 잊기 위한 게이트
- input gate (i) : 현재 정보를 기억하기 위한 게이트
- output gate (o) : 최종 결과를 내보내기 위한 게이트

- 모든 Gate는 시그모이드 함수를 사용하며, 0이라면 아무 것도 넘기지 않고, 값이 1이라면 모든 것을 넘김.
- 히든벡터를 구하기 위해 Cell state를 이용.

```
model = Sequential()
model.add(Embedding(max_features, 128)) # 특성으로 사용할 단어의 수로 임베딩(벡터로 변환)
model.add(LSTM(128, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(1, activation='sigmoid')) # 최종적으로 1개 값 출력
```

koras / keras / src / layers / rnn / lstm.py

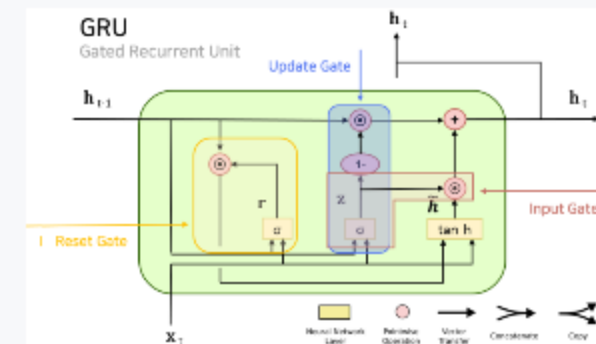
Code Blame 669 Lines (610 loc) - 22.0 KB

226 @keras._context.*Keras.Layers.LSTM

228 class LSTM(Layer):

229 """Long Short-Term Memory Layer - Hochreiter 1997.

GRU



- GRU는 LSTM의 구조를 조금 더 간단하게 개선한 모델
- (f), (i)를 update gate로 통합, (o)를 없애고 reset gate를 정의함.
- reset gate : 이전 hidden state의 값을 얼마나 활용할 것인지에 대한 정보
- update gate : 과거와 현재의 정보를 각각 얼마나 반영할지에 대한 비율을 구하는 것이 핵심.

- GRU에는 기억하는 Cell state는 없고, 시간 방향으로 전파하는 것은 Hidden state 뿐.
- 연산이 적어짐에 따라 LSTM 보다 학습속도가 빠를 수 있음.

```
model = Sequential()
model.add(Embedding(max_features, 128))
model.add(GRU(128, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(1, activation='sigmoid'))
```

v3.3.3 koras / keras / src / layers / rnn / gru.py

Difference 689 Lines (621 loc) - 27.1 KB

10 @keras._context.*Keras.Layers.GRU

11 class GRU(Layer):

12 """Gated Recurrent Unit - Cho et al. 2014.

모델 학습, BERT

시퀀스의 길이가 길어지면 학습이 어려워지는 문제를 해결하기 위해 Transformer 모델이 제안되었고 BERT는 트랜스포머 모델을 기반으로 한 자연어 처리 모델입니다.



- "Bidirectional Encoder Representations from Transformers"의 약자로, 구글에서 개발한 자연어 처리(Natural Language Processing, NLP) 모델.
- 텍스트 분류, 질문 응답, 개체명 인식 등의 다양한 NLP 작업에 적용됨.
- Transformer라는 모델 아키텍처를 기반으로 하며, 이 모델은 특히 언어 모델링 및 NLP 작업에 매우 효과적.
- 사전학습된 BERT는 하나의 추가 output layer를 활용한 fine tuning만으로도 다양한 태스크(자연어추론, 기계독해 등)에서 최고 수준의 성능을 보여줌.



모델 사용



허깅페이스의 transformers 라이브러리

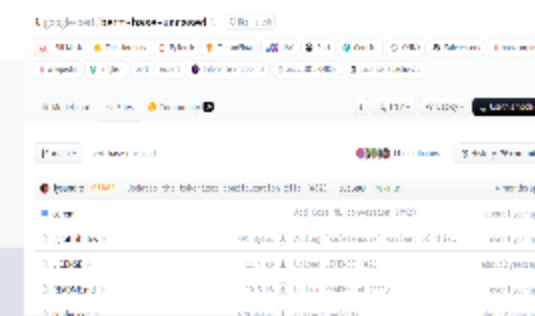
```
!pip install transformers
from transformers import BertModel, BertTokenizer
```

(모델 불러오기)

```
model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertModel.from_pretrained(model_name)
```

(데이터 입력)

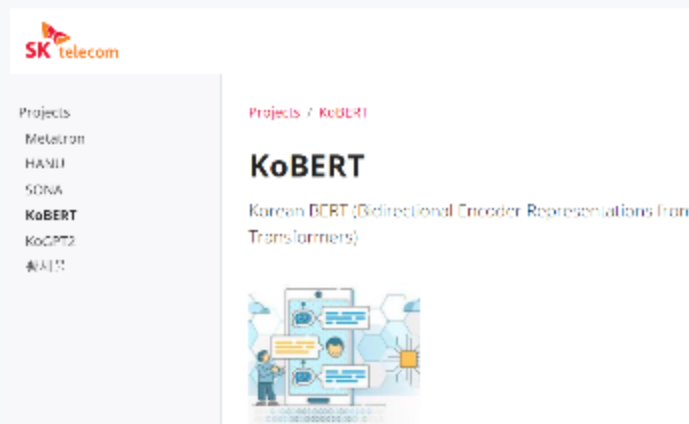
```
text = "Example text to tokenize"
encoded_input = tokenizer(text, return_tensors='pt')
output = model(**encoded_input)
```



- "Attention is All You Need" 논문은 2017년에 발표되었으며, Transformer 모델을 처음으로 소개됨(구글).
- 기계 번역 및 자연어 처리(Natural Language Processing, NLP) 작업에 사용되는 딥러닝 아키텍처
- 예를 들어 텍스트를 분류하는 작업에서는 마지막 인코더 레이어의 출력은 각 클래스에 대한 확률을 예측하고, 질의 응답 작업에서는 디코더를 사용해 입력된 질문과 함께 문맥 정보를 바탕으로 정답을 생성함.

MODELING

skt/kobert

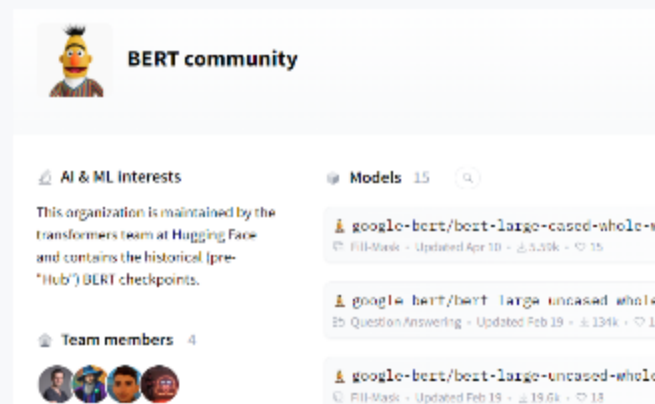


- 위키피디아나 뉴스 등에서 수집한 수백만 개의 한국어 문장으로 이루어진 대규모 말뭉치(corpus)를 학습.
- 한국어의 불규칙한 언어 변화의 특성을 반영하기 위해 데이터 기반 토큰화 (Tokenization) 기법을 적용하여 기존 대비 27%의 토큰만으로 2.6% 이상의 성능 향상을 이끌어 냄.

(모델사용)

```
tokenizer = KoBERTTokenizer.from_pretrained('skt/kobert-base-v1')
bertmodel = BertModel.from_pretrained('skt/kobert-base-v1')
vocab = nlp.vocab.BERTVocab.from_sentencepiece(tokenizer.vocab_file, padding_token='[PAD]')
```

google-bert/bert-base-multilingual-cased

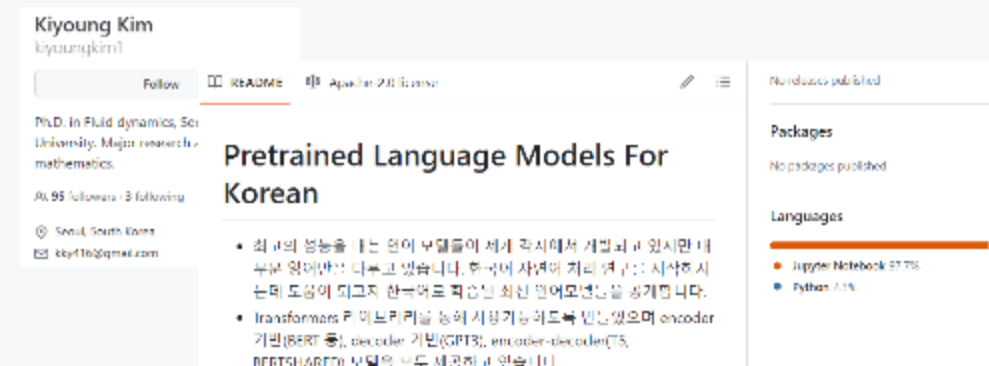


- BERT 논문에서 소개된 다국어 기본모델.
- MLM(Masked Language Modeling) 목표를 사용하여 가장 큰 Wikipedia를 갖춘 상위 104개 언어에 대한 사전 훈련된 모델.
- 각 언어에 대한 전체 Wikipedia 덤프(사용자 및 토론 페이지 제외)가 각 언어에 대한 훈련 데이터로 사용됨.

(모델사용)

```
tokenizer = BertTokenizer.from_pretrained('bert-base-multilingual-cased')
model = BertForSequenceClassification.from_pretrained('bert-base-multilingual-cased', num_labels=2)
# 분류 작업을 위해 미세 조정될 수 있도록 설계됨
```

kykim/bert-kor-base



- 뉴스와 같이 잘 정제된 언어 뿐만 아니라, 실제 인터넷 상에서 쓰이는 신조어, 줄임말, 오자, 탈자를 잘 이해할 수 있는 모델을 개발하기 위해, 대분류 주제별 텍스트를 별도로 수집하였으며 대부분의 데이터는 블로그, 댓글, 리뷰 데이터.
- 불필요하거나 너무 짧은 문장, 중복되는 문장들을 제외하여 100GB의 데이터 중 최종적으로 70GB (약 127억개의 token)의 텍스트 데이터가 학습에 사용됨.

(모델사용)

```
tokenizer = AutoTokenizer.from_pretrained("kykim/bert-kor-base")
model = AutoModelForSequenceClassification.from_pretrained("kykim/bert-kor-base", num_labels=2)
# 텍스트 분류 작업에 적합한 사전 훈련된 모델을 쉽게 불러와서 사용할 수 있음.
```

RESULT

학습결과

분류 모델 성능 평가 지표

		Predicted (예측)		
		Negative (0)	Positive (1)	
Actual (실제)	Negative (0)	True Negative TN	False Positive FP	Specificity (특이도) $\frac{TN}{TN + FP}$
	Positive (1)	False Negative FN	True Positive TP	Recall, Sensitivity (민감도), True positive rate (TPR) $\frac{TP}{TP + FN}$
Accuracy (정확도) $\frac{TP + TN}{TP + TN + FP + FN}$		Precision (정밀도), Positive predictive value (PPV) $\frac{TP}{TP + FP}$		F1-score $= 2 \times \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}$

- 정확도 : 모델이 전체 문제 중에서 정답을 맞춘 비율. (데이터가 불균형할 때(ex) positive:negative=9:1)는 Accuracy만으로 제대로 분류했는지는 알 수 없기 때문에 Recall과 Precision을 사용)
- Precision(정밀도): 모델이 positive라고 예측한 것들 중에서 실제로 정답이 positive인 비율.
- Recall(재현율) = Sensitivity(민감도) = TPR(True Positive Rate): 실제로 정답이 positive인 것들 중에서 모델이 positive라고 예측한 비율.
- F1 score : Recall과 Precision의 조화평균

- 번역모델 : Helsinki-NLP/opus-mt-ko-en

구분		이진분류 (긍정/부정)						다중분류 (긍정/중립/부정)						영어로 번역하여 학습(기본모델)	
		tf-idf+svm	lstm	gru	kobert	google-bert/bert-base-multilingual-cased	kykim/bert-kor-base	tf-idf+svm	lstm	gru	kobert	google-bert/bert-base-multilingual-cased	kykim/bert-kor-base	이진분류	다중분류
1	정확도	0.74	0.74	0.76	0.76	0.73	0.79	0.69	0.70	0.69	0.7	0.71	0.73	0.74	0.69
2	f1score	0.70	0.73	0.73	0.73	0.71	0.77	0.51	0.55	0.54	0.58	0.56	0.61	0.70	0.50
3	정밀도	0.72	0.72	0.73	0.74	0.71	0.78	0.59	0.59	0.58	0.61	0.60	0.63	0.72	0.58
4	재현율	0.69	0.73	0.72	0.73	0.71	0.76	0.49	0.54	0.52	0.57	0.54	0.60	0.68	0.48

모델 활용 예시

마케팅 및 광고

- 소비자의 반응을 이해하고 그에 따른 마케팅 전략을 수립하기 위해 감성분석 모델을 사용할 수 있습니다.

고객 서비스 및 의견 분석

- 감성분석은 온라인 리뷰, 소셜미디어 등에서 고객이 제공하는 피드백을 분석하여 개선점을 찾을 수 있습니다.

금융 및 주식 시장 예측

- 뉴스기사, 소셜 미디어 피드, 트위터 등에서 감성을 분석하여 주식 시장의 움직임을 예측하는데 도움을 줄 수 있습니다.

정치 및 사회 분석

- 뉴스 기사, 소셜 미디어, 온라인 커뮤니티 등에서 정치적 이슈나 사회적 현상에 대한 대중의 의견을 이해하는 데 활용될 수 있습니다.

THANK YOU

감사합니다.

데이터분석 스테디
영화리뷰 감성분석 END